DAO

Desarrollo de Aplicaciones con Objetos

Python • POO • Persistencia • Interfaces • Patrones

UTN – Facultad Regional Córdoba
Ingeniería en Sistemas de Información
Ciclo Lectivo 2026

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA

{ }

```
class DAO:
    def aprender():
        return software
```

Plantilla institucional de clases

Versión 2026

DAO

Plantel Docente

```python
class Alumno:
    def __init__(self, legajo,
nombre):
        self.legajo = legajo
        self.nombre = nombre

    def presentarse(self):
        return self.nombre
```

UTN FRC
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL CÓRDOBA

¿Qué aprenderemos durante el cursado?

El recorrido de la materia está organizado en cuatro grandes ejes, comenzando por los fundamentos del lenguaje Python y avanzando progresivamente hacia el desarrollo de aplicaciones completas utilizando orientación a objetos, interfaces gráficas y persistencia de datos

1 Fundamentos de Python

- Variables
- Tipos de datos
- Funciones
- Archivos

2 Programación Orientada a Objetos

- Clases
- Objetos
- Herencia
- Polimorfismo

3 Interfaces Gráficas

- Eventos
- Formularios
- Validaciones
- Experiencia de usuario

4 Persistencia y Patrones

- SQLite
- JSON
- CSV
- Patrones de diseño

Todo este recorrido culminará con el desarrollo de un proyecto integrador basado en un problema real.

¿Cómo se aprueba la materia?

Parcial

50 %
Evaluación individual.
Recuperatorio disponible.
Nota mínima: 6.

💻 Trabajo Integrador

30 %
Proyecto grupal.
Desarrollo completo de una aplicación.
Presentación y defensa.

Cuestionarios

20 %
Actividades realizadas durante el cursado.
Promedio mínimo: 6.

AD **Aprobación Directa**

✓ Regularizar la materia.
✓ Aprobar todas las instancias.

NF **Nota Final**

50 % Parcial
30 % Proyecto
20 % Cuestionarios

Ejes de aprendizaje

Una visión integrada de desarrollo de software con objetos.

Py **Python**

Lenguaje base para expresar soluciones, automatizar tareas y desarrollar aplicaciones.

OO **Objetos**

Modelado del dominio con clases, responsabilidades, asociaciones y polimorfismo.

DB **Persistencia**

Bases de datos, archivos, transacciones y consistencia de información.

UI **Interfaces**

Pantallas, eventos, formularios, validaciones y experiencia de usuario.

DP **Patrones**

Soluciones reutilizables y criterios de diseño.

Idea central: pasar de ejercicios aislados a soluciones completas, mantenibles y justificadas desde el diseño.

DAO

Material Disponible

Aula Virtual: DAO (2026)

- 4K1: DAO-2026-4K1
- 4K2: DAO-2026-4K2
- 4K3: DAO-2026-4K3

Apunte: Material de la Cátedra

Videos – Tutoriales

Playlist https://www.youtube.com/@DAO-UTN

Presentación de la materia

https://www.youtube.com/watch?v=iA3BR_y1m-E

Uso de entorno de desarrollo

Uso de Visual Studio Code para programar en python - YouTube



PROYECTO INTEGRADOR

De ejercicio a solución

El TP debe integrar análisis, diseño, implementación y justificación.

1 **Análisis**

Dominio, casos de uso, DER y clases

2 **Construcción**

BD, formularios, validaciones y transacciones

3 **Valor**

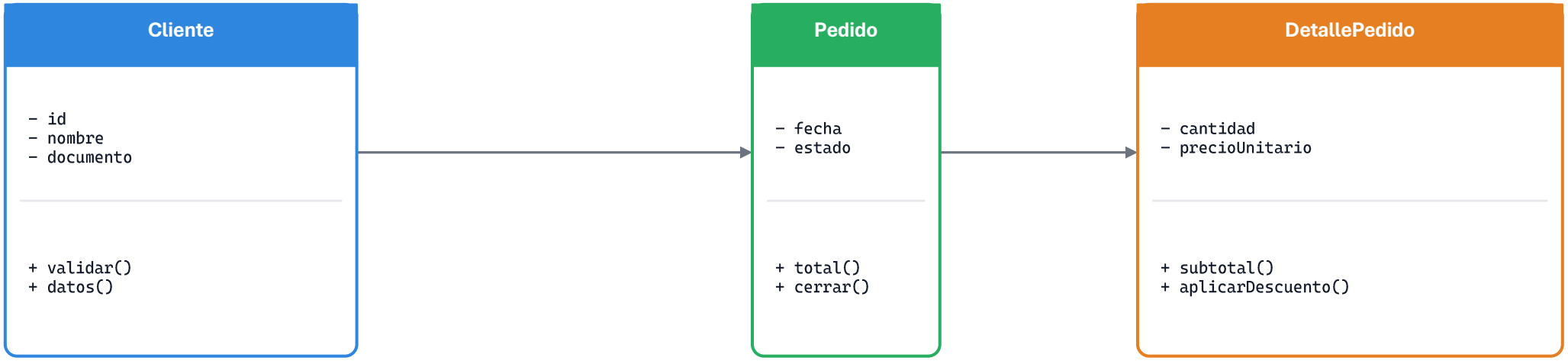
Reportes, gráficos, pruebas y presentación

Durante la presentación final se evalúan tanto el software funcionando como las decisiones de análisis, modelado y diseño.

MODELADO

Diagrama UML simplificado

Usar diagramas para pensar antes de programar.



Sugerencia: mantener el diagrama limpio. No dibujar todo; dibujar lo necesario para explicar el diseño.

Concepto clave: responsabilidad

Una clase no es solo una estructura de datos: representa decisiones de diseño.

Un buen modelo separa responsabilidades.

En DAO buscamos que los objetos colaboren para resolver una situación de negocio.

Cada clase debería tener un propósito claro, atributos coherentes y métodos que representen comportamiento relevante.

Regla práctica: si una clase “hace todo”, probablemente el diseño necesita dividir responsabilidades.

E

Entidad

Representa un concepto importante del dominio.

R

Responsabilidad

Lo que ese objeto sabe hacer.

C

Colaboración

Cómo interactúa con otros objetos.

EJEMPLO GUIADO

Código Python con objetos

Diseño mínimo para explicar encapsulamiento y comportamiento.

Python · POO

```
class Producto:
    def __init__(self, codigo, nombre, precio):
        self.codigo = codigo
        self.nombre = nombre
        self.precio = precio

    def precio_final(self):
        return self.precio

class ProductoImportado(Producto):
    def precio_final(self):
        return self.precio * 1.21
```

Qué observar

El método precio_final() puede invocarse igual, aunque cada clase lo resuelva de manera diferente.

Concepto

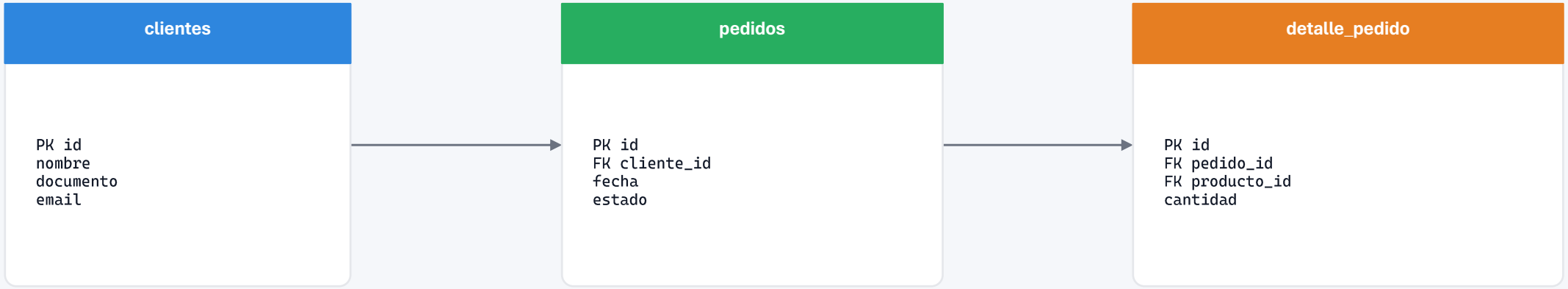
Esto permite introducir polimorfismo sin recurrir a definiciones extensas.

Actividad

Pedir a los estudiantes que agreguen otra subclase con una regla distinta.

Modelo relacional

El diseño de objetos debe dialogar con la base de datos.



PK **Claves primarias**

Identifican registros.

FK **Claves foráneas**

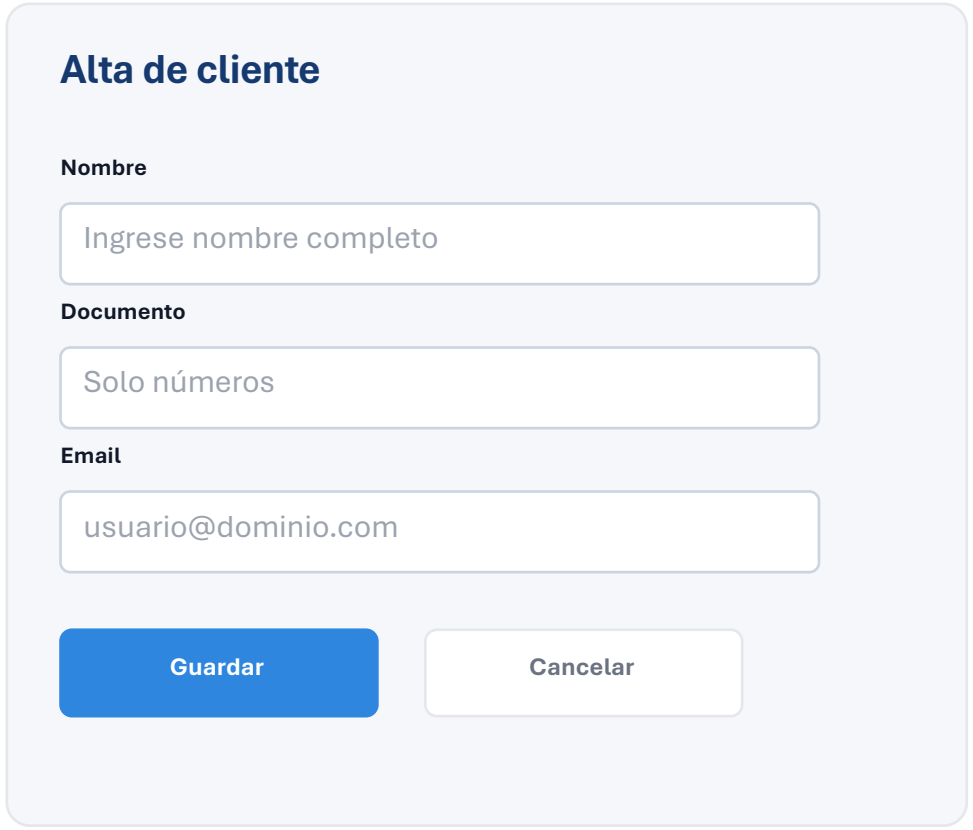
Representan relaciones.

OK **Consistencia**

Evita datos inválidos.

Formulario con validaciones

La pantalla debe guiar al usuario y proteger la consistencia del sistema.



Validaciones obligatorias

Campos requeridos, formatos, unicidad y reglas de negocio.

Mensajes claros

Indicar qué error ocurrió y cómo corregirlo.

Consistencia

No depender solo de la interfaz: validar también en la lógica.

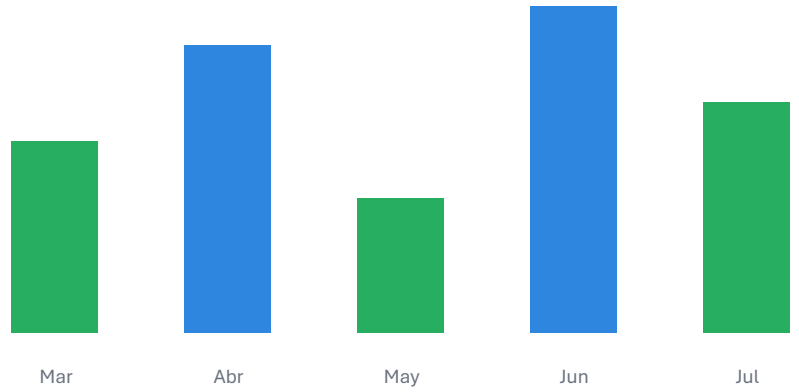
Información para gestión

Los reportes deben responder preguntas del negocio.

Reporte tabular

Cliente	Total
Pérez	$42.000
Gómez	$35.500
Torres	$18.900

Gráfico estadístico



Pregunta guía: ¿qué decisión permite tomar este reporte?

CIERRE

Ideas clave de la clase

Usar esta diapositiva para consolidar los conceptos antes de finalizar.

1

1. Modelar antes de programar

El código refleja decisiones de análisis.

2

2. Diseñar responsabilidades

Las clases deben colaborar, no acumular lógica sin criterio.

3

3. Validar reglas de negocio

La consistencia no es opcional: forma parte del sistema.

Pregunta de salida

¿Qué decisión de diseño tomada hoy cambiarías si el sistema tuviera que crecer durante un año?

DAO

Temas Clase

- Tipos de datos
- programación estructurada
- Estructuras de control
- Secuencias en Python: cadenas, tuplas y listas

